

YourTodo 앱 구조 소개서

1. 한 줄 소개

YourTodo는 개인 할 일 관리, 친구와의 할 일 공유, 캘린더 기반 일정 확인, 홈 화면 위젯, AI 초안 생성을 하나의 Android 앱 안에 통합한 생산성 앱이다.

핵심은 단순한 Todo 목록이 아니라 "내가 해야 할 일"과 "친구에게 받은 일"을 같은 사용자 표면에서 자연스럽게 다루는 것이다. 사용자는 빠르게 개인 Todo를 만들고, 친구에게 할 일을 보내고, 자동수락 관계에서는 수락 절차 없이 할 일이 바로 목록과 캘린더에 반영되는 협업 경험을 얻는다.

2. 제품 경험

영역	사용자 가치	대표 기능
개인 Todo	빠른 기록과 관리	상세 추가, 빠른 추가, 완료 토글, 우선순위, 카테고리, 날짜/시간
Calendar	날짜 중심 확인	월간 캘린더, 선택일 할 일, 오늘 할 일 수, 날짜별 indicator
Calendar Widget	앱 밖에서 일정 확인	홈 화면 월간 위젯, 날짜별 할 일 chip/count, 앱 딥링크
Friends	협업 관계 관리	친구 요청, 친구 목록, 받은/보낸 요청, 친구별 상세
Shared Todo	친구와 할 일 공유	요청형 할 일, 받은 요청 수락/거절, 진행/기록 확인
Direct Assignment	신뢰 관계의 빠른 협업	친구별 자동수락, 자동 할당, Todo/Calendar/Widget 즉시 노출
AI Todo Draft	자연어에서 할 일 초안 생성	한국어 자유 문장 파싱, 다중 task 초안, assignee/date/time/priority 검토
Profile Menu	계정과 앱 설정 진입	계정 정보, 닉네임 복사, 알림 설정, 로그아웃

앱은 사용자가 이미 익숙한 Todo 흐름을 유지하면서도, 친구 협업과 AI 초안 생성을 추가해 "기록 -> 정리 -> 공유 -> 실행" 흐름을 넓힌다.

3. 전체 구조

YourTodo는 기능 모듈과 공통 코어 모듈을 분리한 Android 멀티모듈 구조를 사용한다.

```
app
- 앱 셸, Navigation host, Profile menu
- FCM, notification, WorkManager, widget update 연결
- feature entry와 core runtime 통합

feature::api
- feature route와 외부 공개 계약
```

```

feature:*.entry
- app 셸에 feature를 연결하는 진입 바인딩

feature:*.impl
- Compose 화면, ViewModel, UiState, Action, SideEffect

feature:calendar:widget
- Glance 기반 Android 홈 화면 위젯

core:domain
- use case, repository contract, scheduler contract
- task surface, workspace refresh, reminder recurrence, error model

core:data
- repository 구현
- Room, DataStore, Retrofit data source 조합

core:database / core:datastore / core:network
- Room DB/DAO
- Preferences DataStore와 token 저장 정책
- Retrofit API와 network DTO

core:model / core:ui / core:designsystem / core:testing
- 순수 모델, 공통 UI, 디자인 토큰, 테스트 도구

```

이 구조의 장점은 feature가 data 구현체를 직접 알지 않고 domain use case와 contract만 바라본다는 점이다. 화면 구현은 제품 경험에 집중하고, 데이터 접근과 동기화는 core 계층에서 재사용 가능하게 유지된다.

4. 모듈 구성 장점

4.1 Feature API / Entry / Impl 분리

각 feature는 `api`, `entry`, `impl` 로 나뉜다.

레이어	역할
<code>api</code>	route, 외부 공개 계약, feature identity
<code>entry</code>	app navigation에 feature를 연결하는 바인딩
<code>impl</code>	실제 화면, ViewModel, UI state, user action 처리

이 패턴 덕분에 앱 셸은 feature의 구현 세부사항을 몰라도 된다. 새로운 feature를 추가하거나 기존 feature 내부 구조를 바꿔도 app 쪽 영향이 작다.

4.2 Core 모듈의 명확한 역할

`core:domain`은 앱의 기능 정책을 담고, `core:data`는 저장소 구현을 담당한다. `core:database`, `core:datastore`, `core:network`는 각각 저장소 기술 경계를 맡는다.

이렇게 나누면 use case 테스트는 repository fake로 빠르게 돌릴 수 있고, Room/Retrofit/DataStore는 각자의 모듈에서 집중적으로 검증할 수 있다. Android 앱에서 흔히 생기는 "화면, 네트워크, DB 코드가 한 파일에 섞이는 문제"를 구조적으로 줄인다.

4.3 Compose UDF

화면은 `UiState + Action + SideEffect` 흐름을 따른다.

```
User action
-> ViewModel.onAction(...)
-> use case 호출
-> UiState 갱신
-> Compose rendering
-> navigation/snackbar/dialog는 SideEffect
```

이 패턴은 화면을 예측 가능하게 만든다. View는 상태를 렌더링하고 이벤트를 전달하며, 상태 계산과 사이드 이펙트는 ViewModel/use case 쪽에서 다룬다.

5. 데이터 구조와 동기화

5.1 Local-first Todo

개인 Todo는 Room을 중심으로 빠르게 렌더링된다. 사용자가 Todo를 추가하거나 수정하면 먼저 로컬 DB에 반영되고, 로그인 사용자의 변경은 outbox mutation으로 저장된다.

```
사용자 변경
-> Room todo 즉시 반영
-> outbox mutation 저장
-> UI는 Room Flow로 즉시 갱신
-> 네트워크 가능 시 서버와 동기화
```

이 방식은 오프라인이나 느린 네트워크에서도 사용자가 작업을 이어갈 수 있게 한다.

5.2 Outbox + Cursor Sync

Todo sync는 pull/push/pull 흐름을 따른다.

```
pull(cursor)
-> 서버 변경 수신
-> local pending 상태와 병합
push(outbox)
-> 로컬 변경 전송
-> mutation 결과 반영
pull(latest cursor)
-> 서버 기준 최신 상태로 수렴
```

서버는 최종 진실의 원천이고, Android Room은 화면 캐시와 미전송 변경 저장소 역할을 함께 수행한다.

5.3 Task Surface

Todo, Calendar, Widget은 모두 "사용자가 보는 할 일 표면"을 공유한다. 개인 Todo와 친구에게 받은 Assigned Todo는 `TaskSurface` domain 정책으로 합쳐지고, 화면은 이 결과를 각자 UI 모델로 변환한다.

소비 표면	공유 정책
Todo list	필터, 정렬, 섹션, 완료 상태
Calendar	날짜별 indicator, 선택일 할 일
Widget	월간 count/chip

이 덕분에 Todo 화면과 Calendar, Widget이 서로 다른 기준으로 할 일을 보여주는 일을 줄이고, 같은 사용자 현실을 여러 표면에서 일관되게 표현한다.

6. 협업 기능 구조

6.1 Friends

Friends 기능은 서버 계정 기반의 관계 관리 표면이다. 친구 목록, 받은 요청, 보낸 요청을 서버에서 가져오고, 네트워크 실패 시 사용자가 상태를 이해할 수 있도록 unavailable/stale UX를 제공한다.

6.2 Assignment

친구에게 보낸 할 일과 친구에게 받은 할 일은 Assigned Todo 모델로 관리된다. 서버가 진실의 원천이고 Android는 Room cache를 통해 받은/보낸 feed를 빠르게 보여준다.

Feed	의미
Pending	수락/거절 대기
Active	진행 중 또는 수락됨
History	완료, 거절, 취소 기록

Assignment feed에는 refresh timestamp가 있어 화면이 최신성을 판단할 수 있다.

6.3 Direct Assignment

친구별 자동수락 설정을 켜면 해당 친구가 보낸 할 일이 수락 단계 없이 바로 Todo surface에 들어온다. 사용자는 신뢰하는 관계에서는 빠르게 협업하고, 그렇지 않은 관계에서는 기존 요청/수락 흐름을 유지할 수 있다.

자동 할당된 항목은 Todo, Calendar, Widget, Friends 상세에 같은 방식으로 반영된다.

7. AI Todo Draft 구조

AI Todo Draft는 사용자의 자연어 문장을 바로 저장하지 않는다. 먼저 초안을 만들고, 사용자가 검토한 뒤 저장한다.

자연어 입력

-> ParseAiTodoDraftsUseCase

-> AiTodoDraftRepository

-> AI proxy

-> structured draft result

-> editable review sheet

-> selected draft 저장

이 구조는 AI의 편리함과 사용자의 통제권을 함께 제공한다. AI는 제목, 담당자, 날짜, 시간, 우선순위를 제안하고, 사용자는 저장 전에 모든 항목을 확인할 수 있다.

8. Android 런타임 통합

런타임 요소	앱에서의 역할
Hilt	repository/use case/ViewModel DI
Room	Todo, Category, Reminder, Assignment cache
DataStore	Auth session, sync cursor, UI preference, push token, freshness
Retrofit/OkHttp	Auth, Todo sync, Friends, Assignments, Push, AI proxy
WorkManager	Todo reminder scheduling
Firebase Messaging	Push token과 서버 이벤트 수신
Glance	Calendar home widget
Navigation Compose	feature route와 app shell 연결

런타임 기술은 기능별로 직접 얹히지 않고 domain contract를 통해 연결된다. 예를 들어 widget update는 domain `CalendarWidgetUpdater` 계약으로 추상화되어 Todo 변경이나 workspace refresh가 Glance 구현을 직접 알 필요가 없다.

9. 보안과 계정 관리

Auth token은 Android Keystore 기반 AES-GCM 암호화 값을 DataStore에 저장한다. legacy plaintext token은 migration 호환을 위해 읽을 수 있지만, 성공적으로 암호화 저장되면 제거된다.

로그아웃은 push token 삭제, 사용자 범위 로컬 데이터 삭제, sync state 정리, auth session 삭제 순서로 진행된다. 계정 전환 시 이전 사용자의 서버 데이터가 현재 화면에 남지 않도록 정리 경계를 둔다.

10. 테스트와 품질 기반

프로젝트는 기능별 단위 테스트와 Android 계층 테스트를 고려한 구조다.

테스트 영역	예시
Domain use case	TaskSurface, WorkspaceRefresh, Assignment use cases
Data repository	Todo sync, Assignment cache, Auth cleanup, Reminder
Database	DAO, migration, query behavior
DataStore	token policy, preferences migration
Network	DTO serialization, API data source
Feature ViewModel	Todo, Calendar, Friends, Auth, AI draft
Widget	presenter, date grid, content state

테스트 외에도 저장소 자체에 품질을 지키는 하네스가 포함되어 있다.

하네스	앱 품질에 주는 효과
GitHub Actions Android CI	PR/push마다 단위 테스트, 디버그 빌드, strict lint, coverage verification을 실행
<code>.husky pre-commit/pre-push</code>	main 직접 commit/push를 막고, 최신 main 기준 작업과 영향 모듈 lint를 확인
Product harness check	모듈별 <code>AGENTS.md</code> , 루트 모듈 인덱스, Gradle 의존 방향, 금지 import를 검사
<code>coverageVerifyAll</code>	core domain/data/database와 Todo ViewModel 계층의 non-view 로직 커버리지 기준을 검증
Strict Android lint	warning을 error로 다루고 dependency lint까지 확인해 Android 품질 문제를 조기 발견
TDD Guard hook	운영 코드 변경 전에 같은 모듈 테스트 변경을 요구해 테스트 선행 흐름을 강화
PR template	문제, 범위, 설계, 테스트, 커버리지, 리스크, 마이그레이션을 PR마다 구조화
<code>AGENTS.md</code> 정책	모듈 경계, 테스트 기준, 리소스/마이그레이션/리뷰 규칙을 일관되게 적용

테스트하기 좋은 경계가 곧 유지보수하기 좋은 경계다. YourTodo는 ViewModel/use case/repository/DAO가 분리되어 있어 기능 변경 시 영향 범위를 좁혀 검증할 수 있다.

11. 앱 구성의 강점

강점	설명
확장 가능한 멀티모듈 구조	feature와 core가 분리되어 기능 추가와 리팩터링에 유리
Local-first 경험	Todo는 Room 기반으로 빠르게 반응하고 서버와 수렴
협업형 Todo 모델	친구 요청, shared todo, direct assignment를 자연스럽게 연결
일관된 task surface	Todo, Calendar, Widget이 같은 domain 정책을 공유
사용자 검토형 AI	AI 초안을 바로 저장하지 않고 review sheet를 거침
안전한 인증 저장	Keystore 기반 token 저장 정책
런타임 통합력	Push, WorkManager, Widget, Navigation이 domain contract로 연결
테스트 가능한 설계	use case와 repository contract 중심의 검증 구조
자동화 품질 하네스	hook, CI, lint, coverage, PR template이 변경 품질을 단계별로 확인